DAY - 13 SUMMARY

Inheritance in Java

Inheritance:

Inheritance is an Object-Oriented Programming (OOP) concept that allows one class to acquire the properties and methods of another class.

- The existing class is called Parent Class (Super Class).
- The new class is called Child Class (Sub Class).
- It helps in code reusability and reduces duplication.

Syntax

```
class Parent {
   // properties and methods
}

class Child extends Parent {
   // additional properties and methods
}
```

Types of Inheritance in Java

1. Single Inheritance

One child class inherits from one parent class.

Example

```
class Animal {
   void eat() {
      System.out.println("Animal is eating");
   }
}

class Dog extends Animal {
   void bark() {
      System.out.println("Dog is barking");
   }
}

public class Main {
   public static void main(String[] args) {
      Dog d = new Dog();
      d.eat();
      d.bark();
```

```
    }
}
```

Output:

Animal is eating
Dog is barking

2. Multilevel Inheritance

A class inherits from another class, which is already inherited from another class.

Example

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Puppy is weeping");
    }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.weep();
    }
}
```

Output:

Animal is eating
Dog is barking
Puppy is weeping

3. Hierarchical Inheritance

Multiple child classes inherit from the same parent class.

Example

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat is meowing");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        Cat c = new Cat();

        d.eat();
        d.bark();

        c.eat();
        c.meow();
    }
}
```

Output:

```
Animal is eating
Dog is barking
Animal is eating
Cat is meowing
```

4. Multiple Inheritance (Using Interfaces)

Java does not support multiple inheritance through classes to avoid ambiguity. It can be achieved using interfaces.

Example

```
interface A {
    void show();
}

interface B {
    void display();
}

class Test implements A, B {
    public void show() {
        System.out.println("Interface A");
    }

    public void display() {
        System.out.println("Interface B");
    }
}

public class Main {
    public static void main(String[] args) {
        Test t = new Test();
        t.show();
        t.display();
    }
}
```

Output:

```
Interface A
Interface B
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more inheritance types. Java supports it only through interfaces.

Example

```
interface A {
    void show();
}

interface B extends A {
    void display();
}
```

```java
class Test implements B {
    public void show() {
        System.out.println("Show Method");
    }

    public void display() {
        System.out.println("Display Method");
    }
}

public class Main {
    public static void main(String[] args) {
        Test t = new Test();
        t.show();
        t.display();
    }
}
```